Faculty of Engineering

Cairo University

Faculty of Engineering

Electronics and Electrical Communications Engineering Department

Third Year

Analog Communications

MATLAB implementation of a superheterodyne receiver

Name	سهيله احمد محمد عبدالمجيد
Section	2
BN	67
ID	9220377

Table of Contents

1. Transmitter 4

Discussion 4

Figures 4

2. RF stage 5

Discussion 5

Figures 5

3. Mixer 6

Discussion 6

Figures 6

4. IF stage 7

Discussion 7

Figures 7

5. Baseband detection 8

Discussion 8

Figures 8

6. The role of the RF, the IF, and the baseband detector. 9

7. Use the command 'sound' on the demodulated signal Please comment about this step in your report 9

8. Add "noise" to your signal and then play "sound" the signal. 9

Discussion: 9

Figures: 9

9. Repeat parts 2 and 3 but after removing the RF BPF. 11

Discussion 11

Figures 11

10. the receiver oscillator has frequency offset by 0.2 KHz and 1.2 KHz? 13

Discussion 13

Figures 13

The code 15

Table of figures

Figure 1: Modulated signals in frequency 4
Figure 2: Modulated signals in time 4
Figure 3: RF signals in time.....5
Figure 4: RF signals in frequency.....5
Figure 5: Signals after mixer in time 6
Figure 6: Signals after mixer in frequency 6
Figure 7: IF signals in frequency 7
Figure 8: IF signals in time 7
Figure 9: Signals after LPF in frequency.....8
Figure 10: Signals after LPF in time.....8
Figure 11: Demodulated baseband signals in frequency 8
Figure 12: Demodulated baseband signals in time 8
Figure 13: signal in time and frequency before noise.....9
Figure 14: signal in time and frequency after noise 10
Figure 15: signal in time and frequency after removing RF stage 12
Figure 16: signal in time and frequency with oscillator offset = 0.2 KHz 13
Figure 17: signal in time and frequency with oscillator offset = 1.2 KHz 14

1. Transmitter

Discussion

1. Prepares the Input Signals:
   - Stereo to mono conversion.
   - Signal equalization.
2. Upsamples the signals: The signals are upsampled to increase the sampling frequency and avoid **aliasing** during modulation.
3. Performs AM modulation: The system uses **Double-Sideband Suppressed Carrier (DSB-SC)** amplitude modulation.
4. Adds the two modulated signals.

The signals are read using audioread() and then sampled to satisfy Nyquist criteria ($Fs \geq 2Fc$). Then both signals are modulated to different frequencies (100KHz, 150KHz) to be able to transmit them over the channel (air). We add the two signals by padding the smallest with zeros to make them equal in length. Modulation is done for many reasons such as having an antenna of smaller size and performing FDM.

Figures

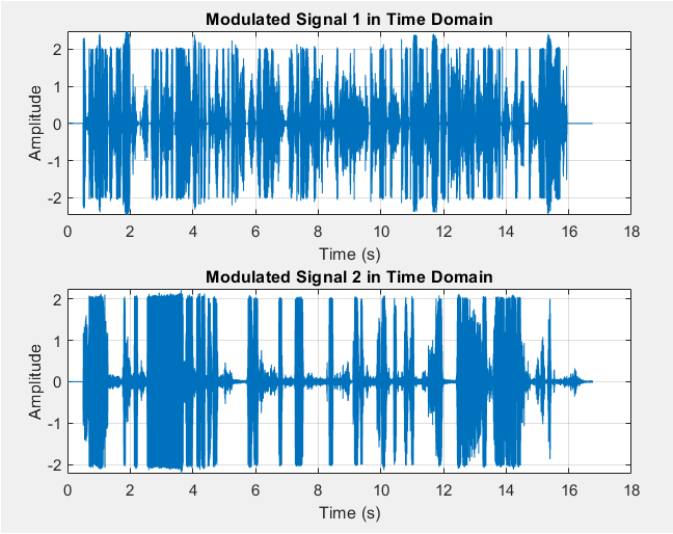


Figure 2: Modulated signals in time

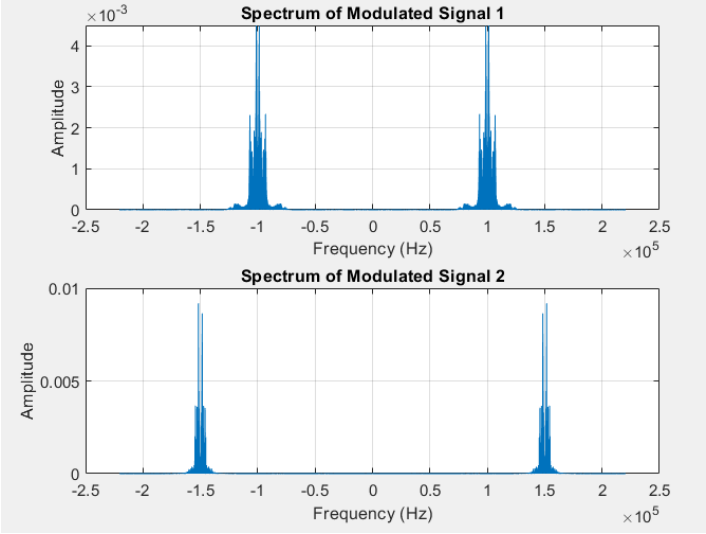


Figure 1: Modulated signals in frequency

2. RF stage

Discussion

As mentioned in the project statement: RF band pass filter is used to reject the interference images or other signals that appear during demodulation. The filter used is with BW = 20KHz and of the fourth order.

Figures

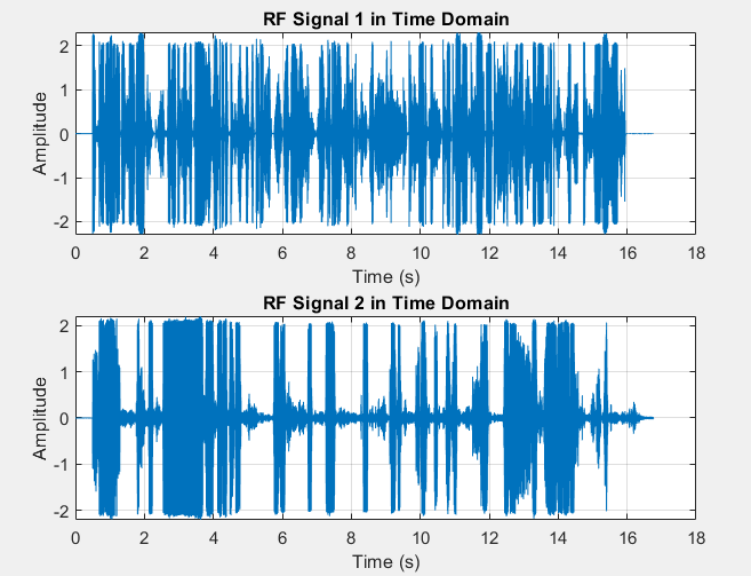


Figure 3: RF signals in time

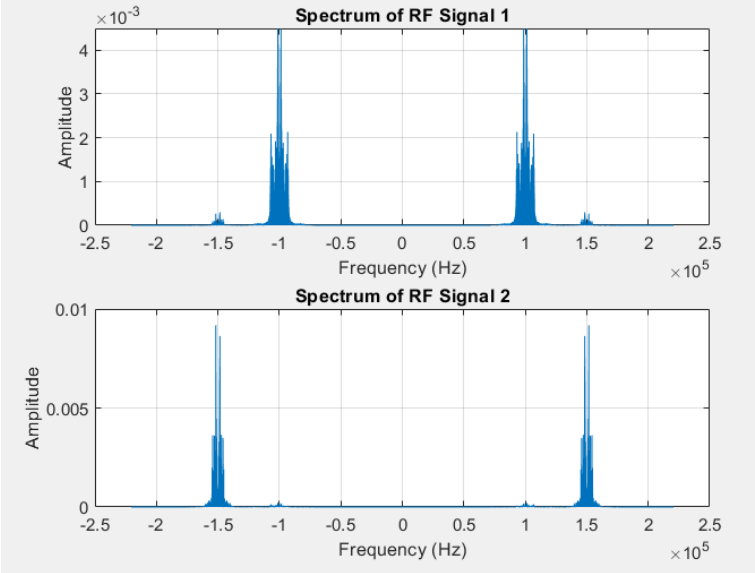


Figure 4: RF signals in frequency

3. Mixer

Discussion

The mixer is used to generate a carrier frequency ωc + ω_{IF} to multiply it by the output of the RF stage. The purpose of using a mixer and oscillator is primarily for frequency translation, which is essential in various stages of signal processing. The frequency translation can be upconversion (modulating) or downconversion (demodulating).

Figures

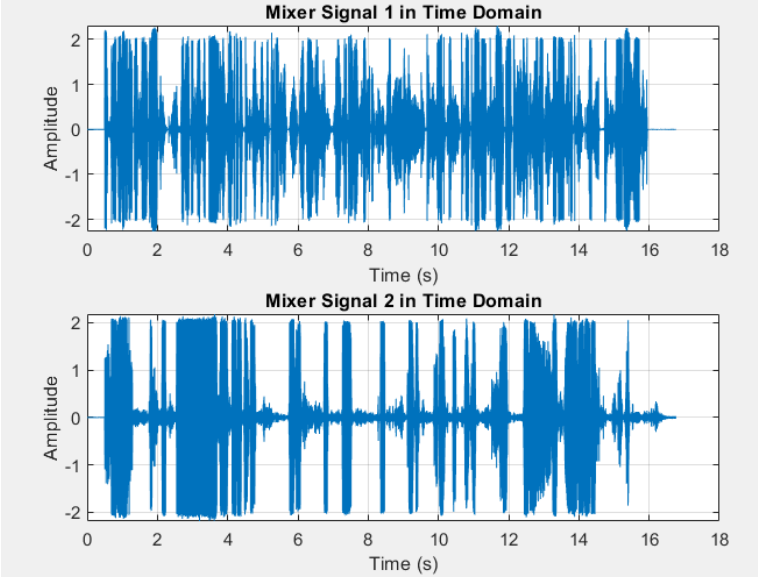


Figure 5: Signals after mixer in time

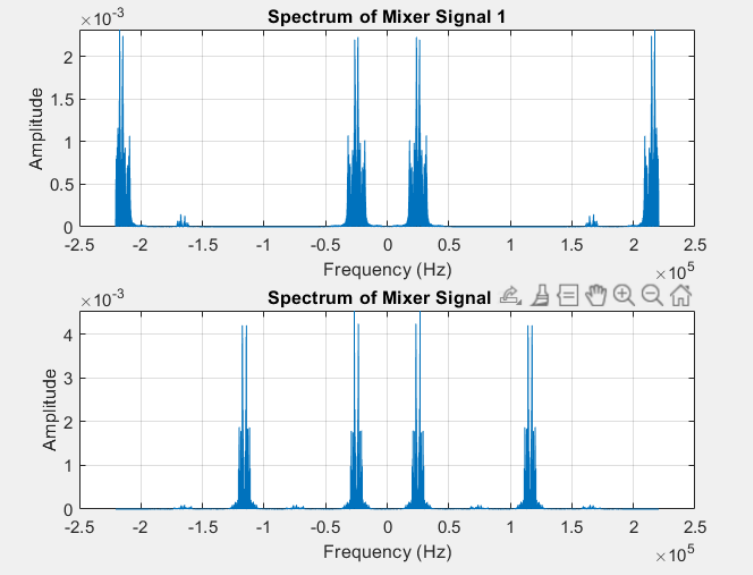


Figure 6: Signals after mixer in frequency

4. IF stage

Discussion

This stage is used to remove or reject the alias (either at high or low frequencies) seen in the last figure due to demodulation. This stage is a bit similar to the RF stage as we use a BPF with BW = 100 kHz and of the fourth order.

Figures

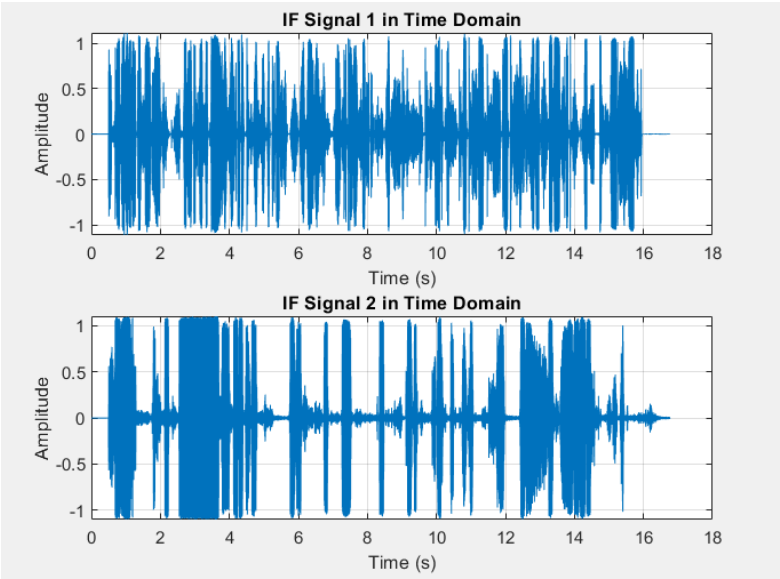


Figure 8: IF signals in time

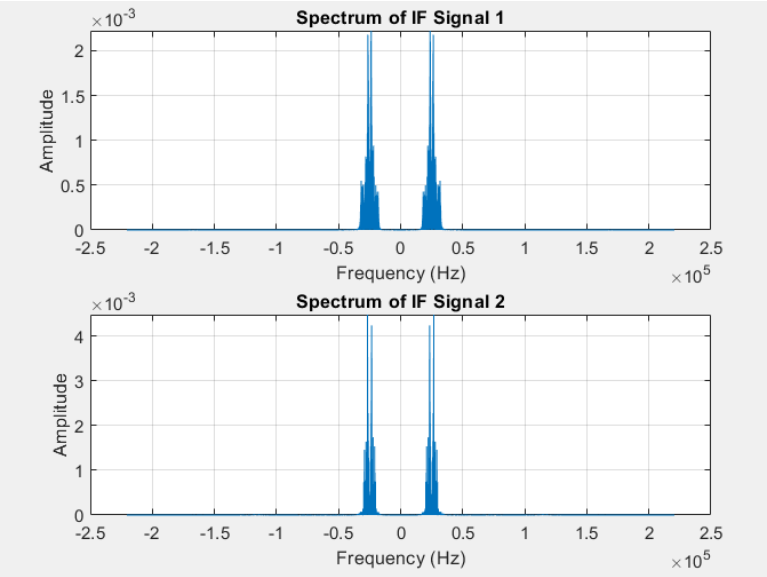


Figure 7: IF signals in frequency

5. Baseband detection

Discussion

This stage consists of two steps:

- Demodulation: by multiplying the two signals by carriers to shift them to the baseband.
- Filtering using an LPF: to reject the alias at the higher and lower frequencies.

Figures

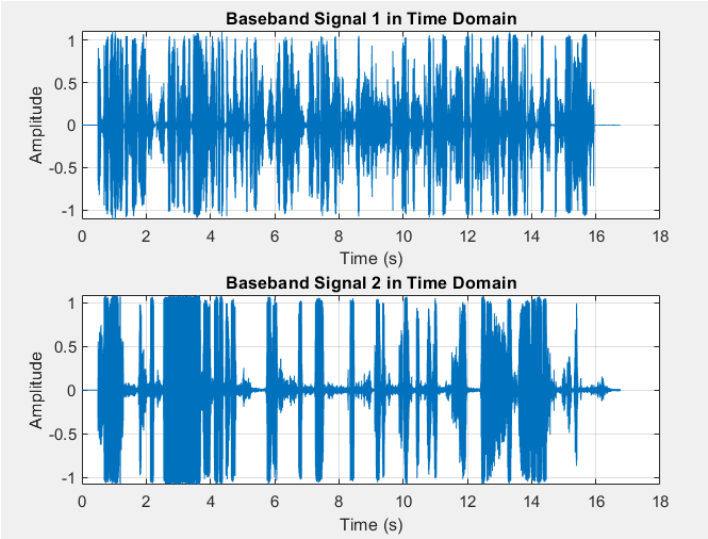


Figure 12: Demodulated baseband signals in time

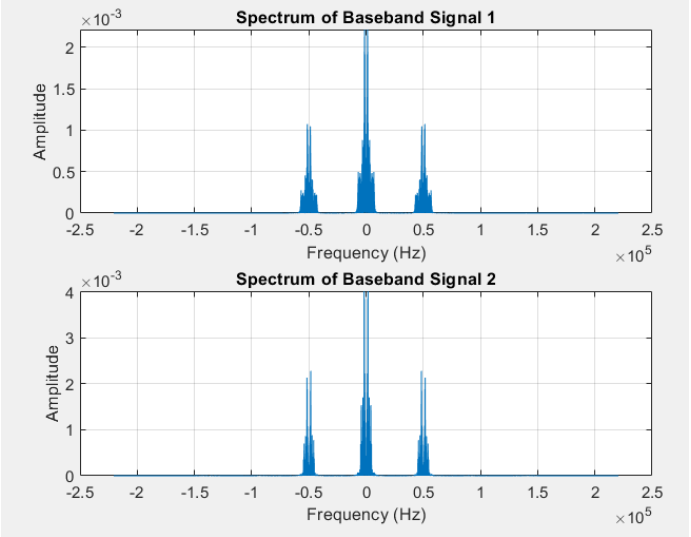


Figure 11: Demodulated baseband signals in frequency

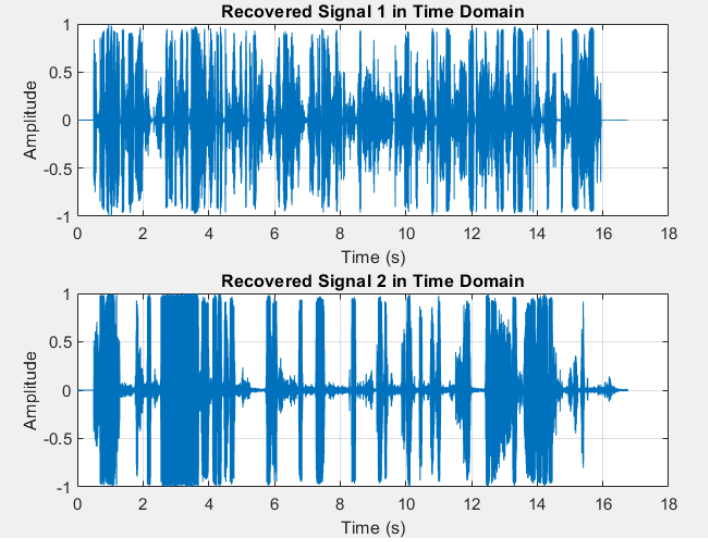


Figure 10: Signals after LPF in time

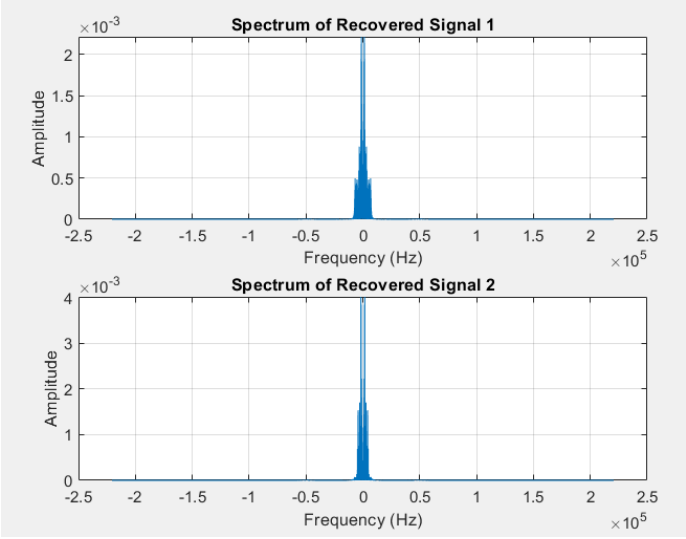


Figure 9: Signals after LPF in frequency

Part b questions

6. The role of the RF, the IF, and the baseband detector.

The RF stage filters the received high-frequency signal to isolate the desired signal from noise and interference (images). The IF stage translates the RF signal to an intermediate frequency where filtering is easier and more selective, improving the system's performance and simplifying the hardware design. The baseband detector extracts the original information (audio or data) from the IF signal.

The IF stage is crucial because it allows for better signal processing with more stable and effective filters at fixed intermediate frequencies.

7. Use the command 'sound' on the demodulated signal Please comment about this step in your report

Using the function `sound` I could successfully listen to the signal without any noise or interference

8. Add "noise" to your signal and then play "sound" the signal.

Discussion:

After adding noise to the monophonic signals it was realized that:

- In time: The noisy signals will show irregular, random fluctuations (caused by the Gaussian noise), making them look distorted and less smooth.
- In frequency: The spectrum of the noisy signal will have additional energy spread across all frequencies (a flat or widespread noise floor due to the Gaussian noise).
- Using the function sound: the noisy sound will have an audible "hiss" or "static" overlaid on the original signal. The severity of the noise depends on the Signal-to-Noise Ratio (SNR):
    - High SNR (e.g., 30 dB): The noise will be subtle.
    - Low SNR (e.g., 5 dB): The noise will dominate, making the original signal hard to hear.

Figures:

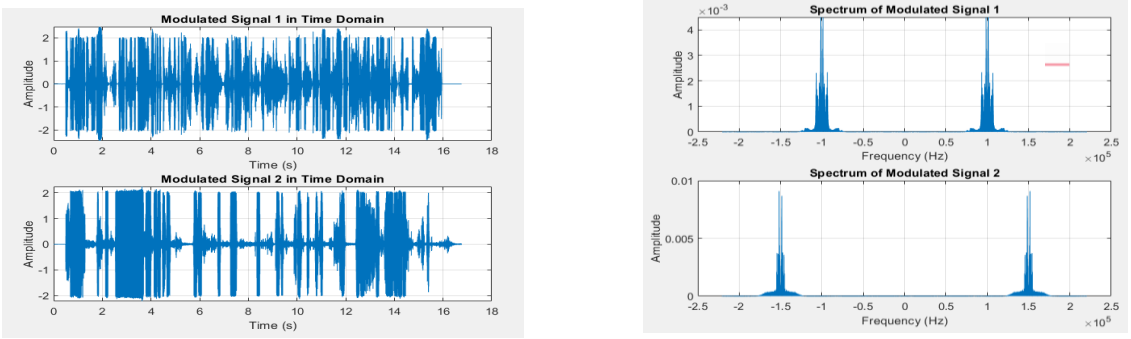
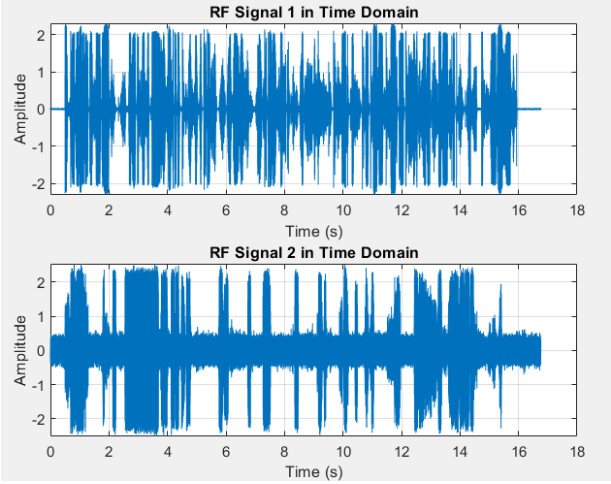


Figure 13: signal in time and frequency before noise



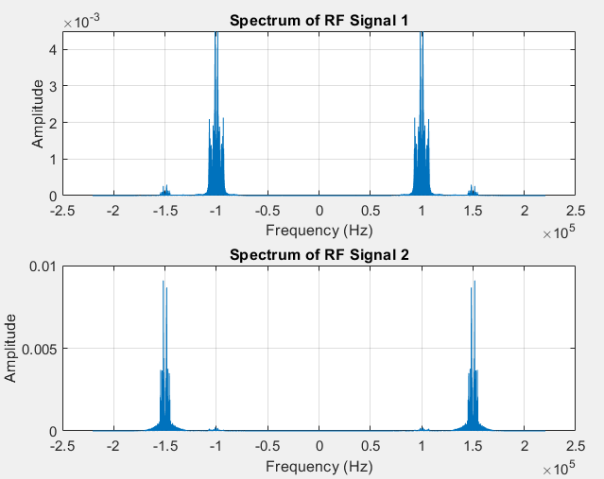
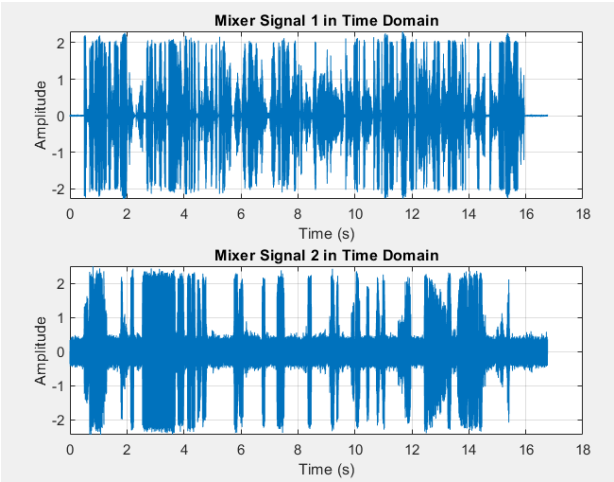
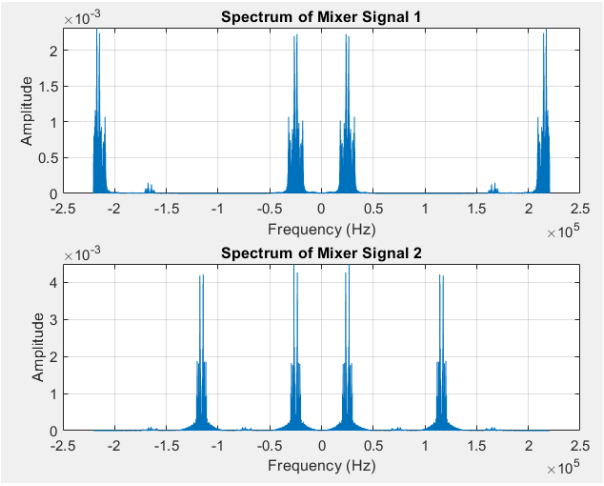
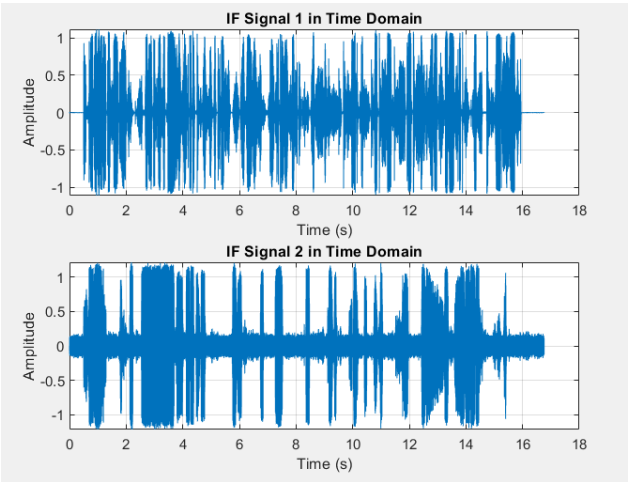
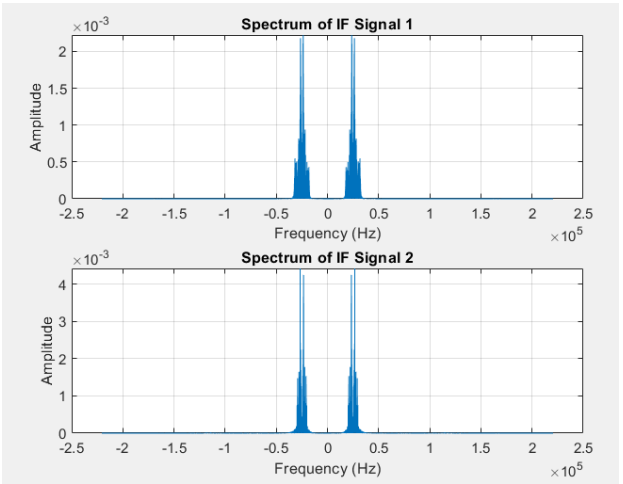


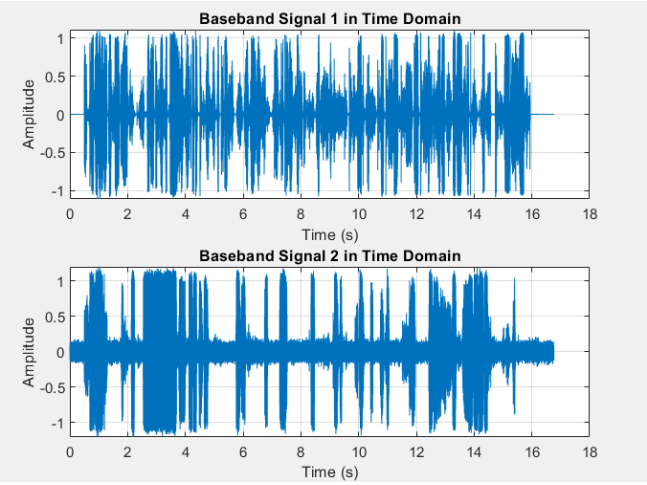
Figure 14: signal in time and frequency after noise

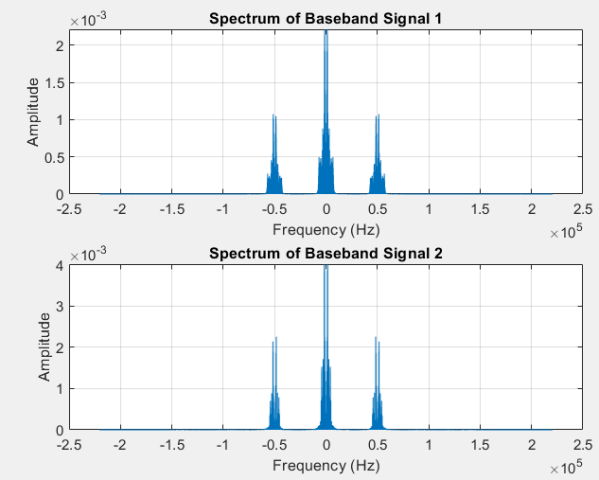


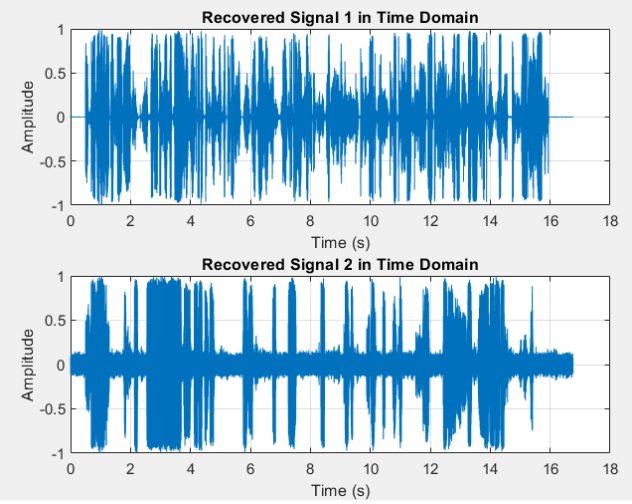


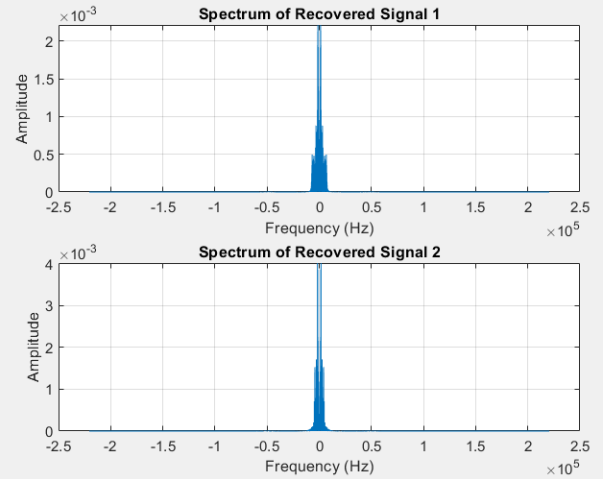












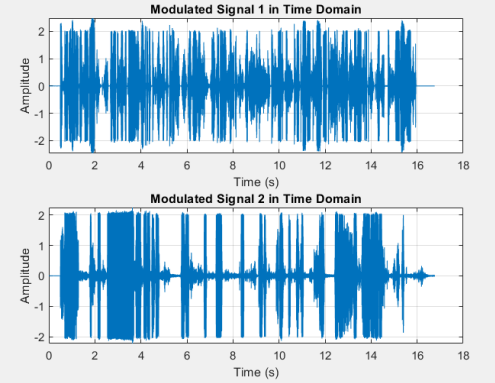
9. Repeat parts 2 and 3 but after removing the RF BPF.

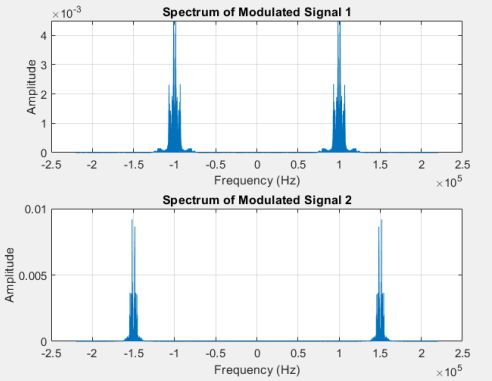
Discussion

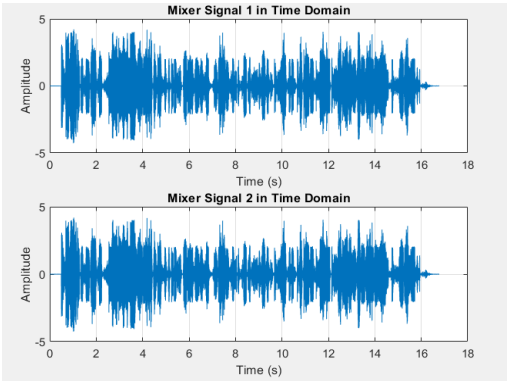
After removing the RF stage and trying to demodulate:

- Using the function sound: now we can't the signal clearly as it interfered with the other signal (because removing the BPF introduced IF images for both signals)

Figures







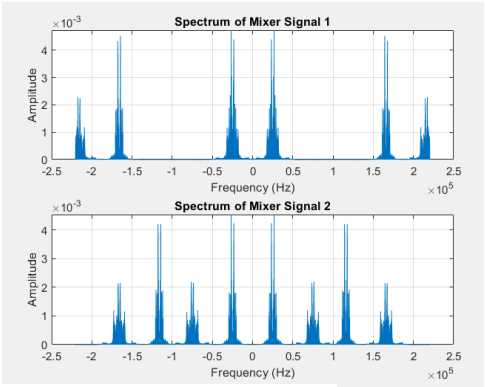
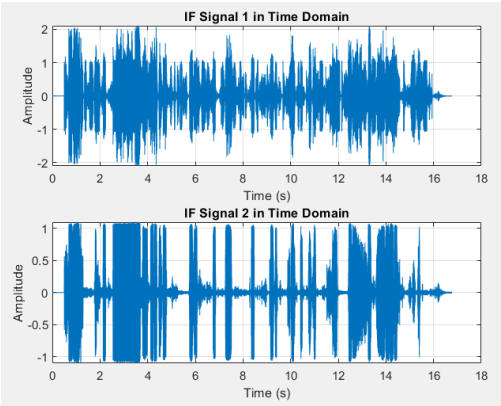
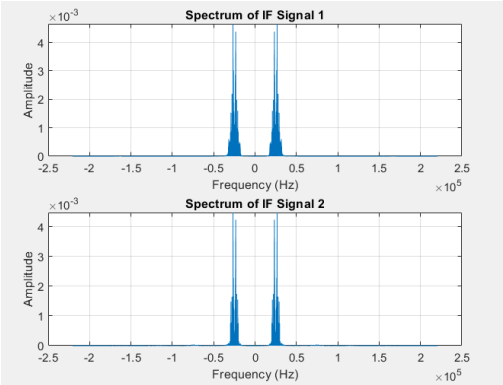
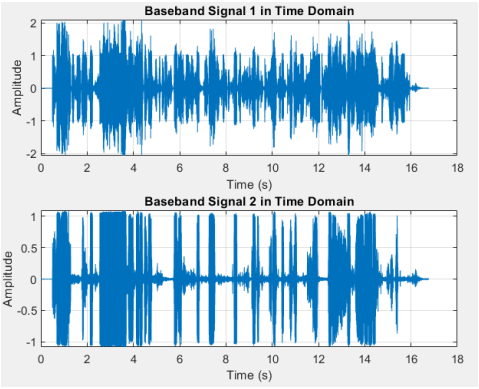
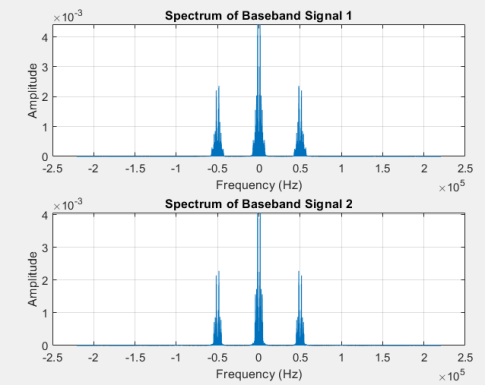


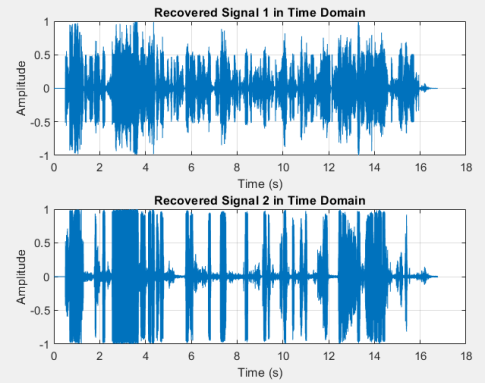
Figure 15: signal in time and frequency after removing RF stage

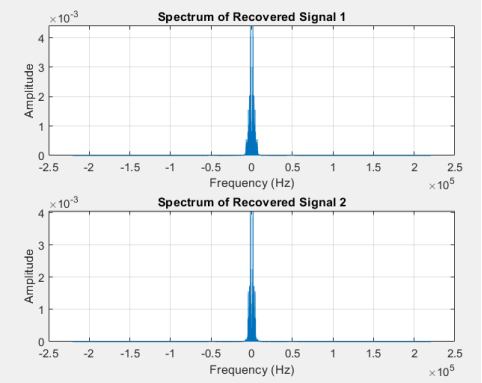












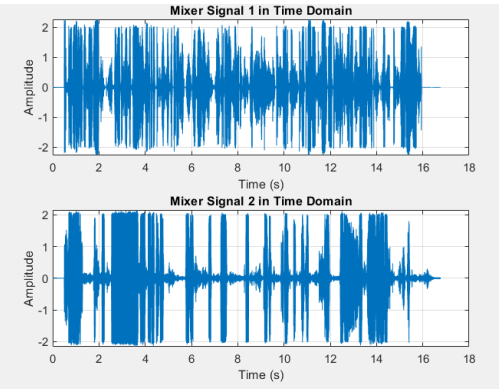
10. the receiver oscillator has frequency offset by 0.2 KHz and 1.2 KHz?

Discussion

- A slight offset (eg: 0.2 KHz) may not significantly distort the signal but will result in a slight pitch change in the recovered audio. This happens because the frequency content of the signal is shifted.
- A larger offset (eg: 1.2 KHz) can lead to noticeable distortion: The pitch of the audio will change more significantly, making the audio sound unnatural. In severe cases, the offset might cause part of the spectrum to shift out of the filter's bandwidth, leading to a loss of parts of the signal or complete unintelligibility.
- Using the function sound: you can hear a strange sound (pitch) with the signal that increases and decreases the sound quality severely when increasing the offset.

Figures

Case 1: offset = 0.2 KHz



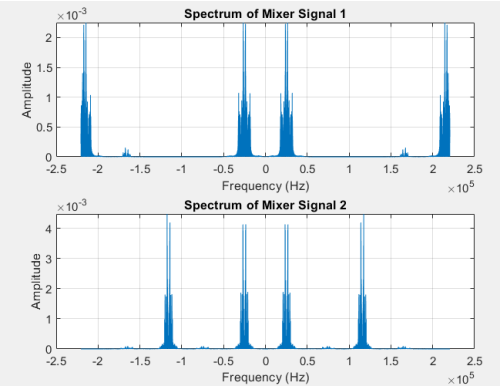
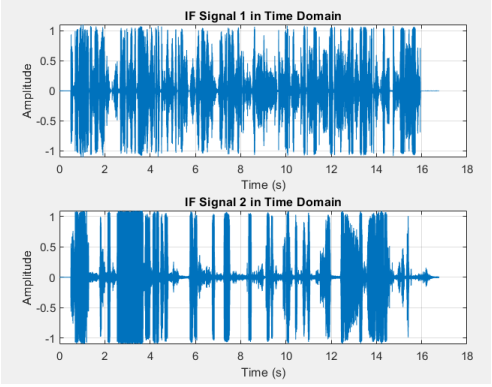
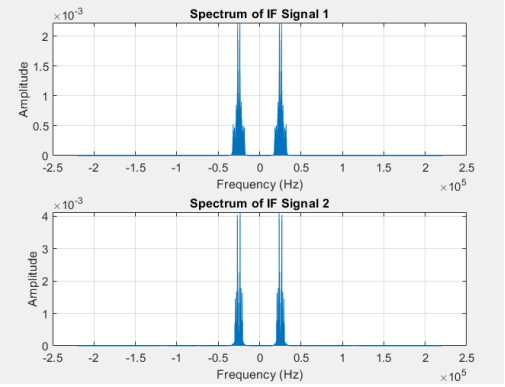
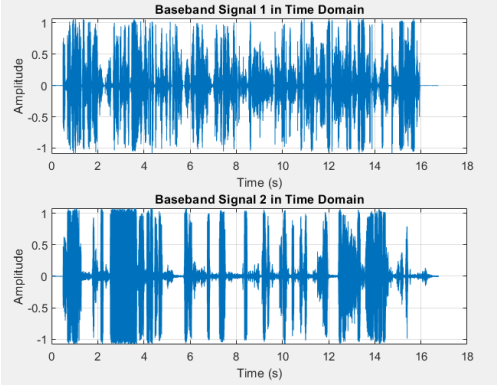
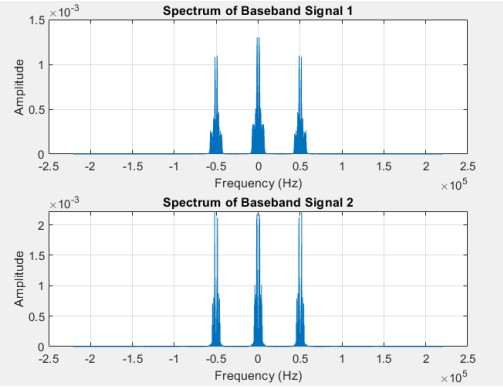


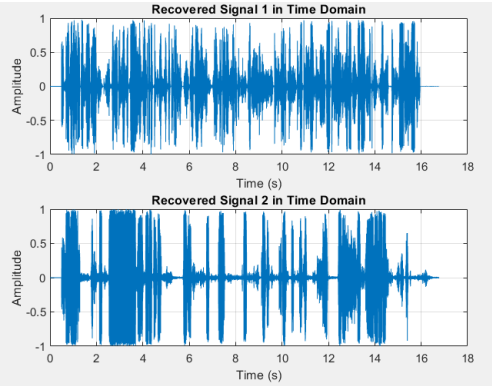
Figure 16: signal in time and frequency with oscillator offset = 0.2 KHz

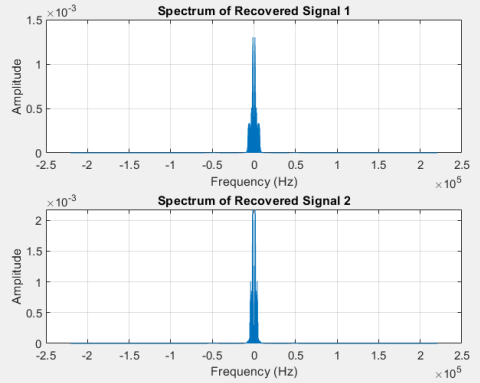




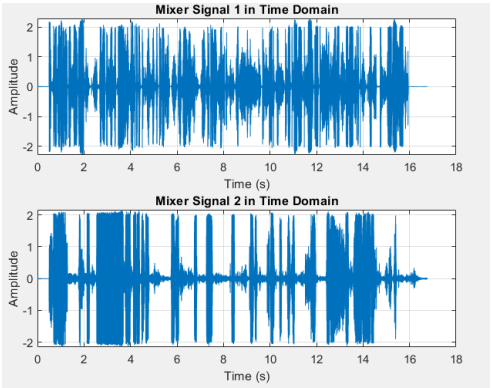








Case 2: offset = 1.2 KHz



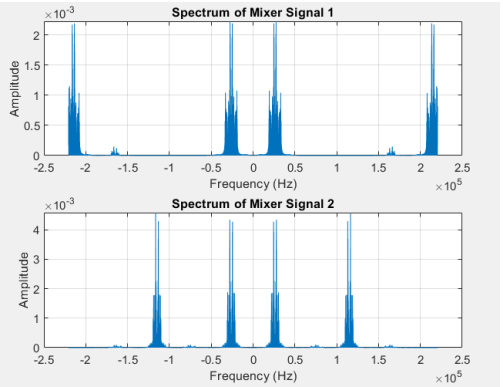
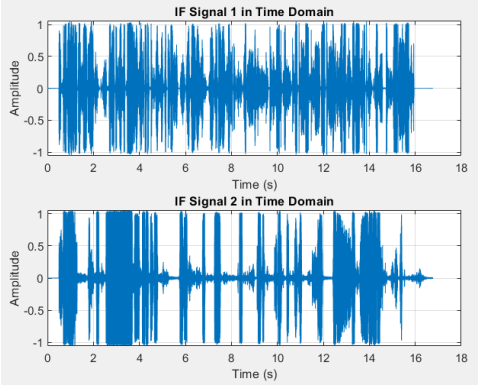
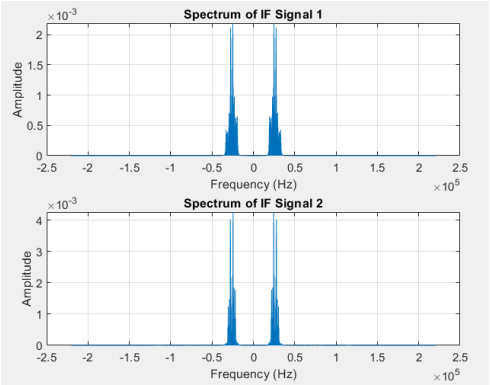
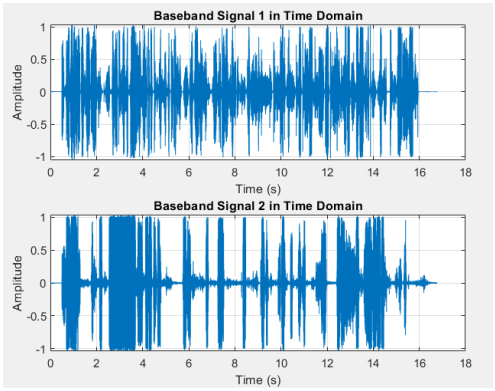
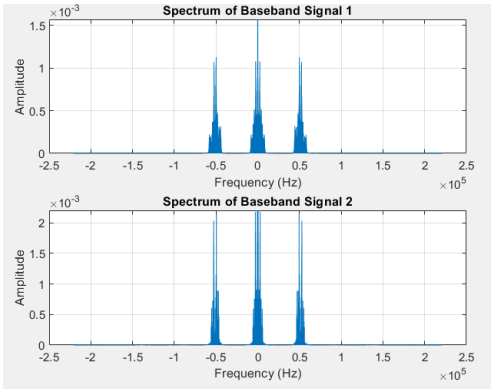


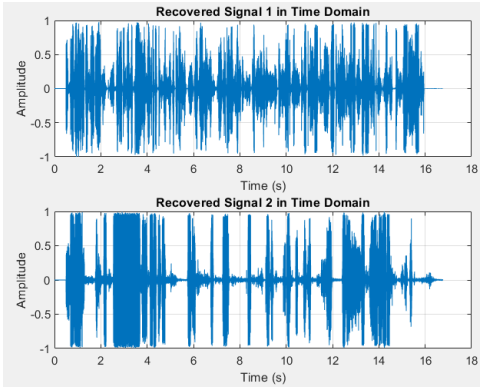
Figure 17: signal in time and frequency with oscillator offset = 1.2 KHz

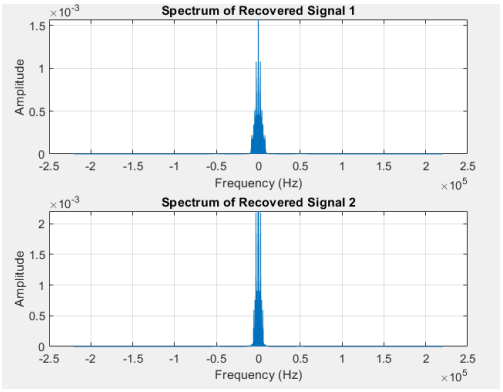












The code

```
% Read input audio files
[signal1, Fs1] = audioread('Short_RussianVoice.wav'); % Signal 1
[signal2, Fs2] = audioread('Short_QuranPalestine.wav'); % Signal 2

% Ensure both signals have the same sampling frequency (resample if
necessary)
if Fs1 ~= Fs2
    Fs = max(Fs1, Fs2); % Use the highest sampling frequency
    signal1 = resample(signal1, Fs, Fs1);
    signal2 = resample(signal2, Fs, Fs2);
else
    Fs = Fs1; % Use the same sampling frequency
end


% Convert stereo signals to mono
signal1_mono = sum(signal1, 2); % Sum stereo channels for Signal 1
signal2_mono = sum(signal2, 2); % Sum stereo channels for Signal 2
```

```
% Plot original signals in the time domain
figure;
subplot(2, 1, 1);
plot((0:length(signal1_mono)-1) / Fs, signal1_mono);
xlabel('Time (s)');
ylabel('Amplitude');
title('Original Signal 1 in Time Domain');
grid on;

subplot(2, 1, 2);
plot((0:length(signal2_mono)-1) / Fs, signal2_mono);
xlabel('Time (s)');
ylabel('Amplitude');
title('Original Signal 2 in Time Domain');
grid on;

% Compute and plot the spectrum of the original signals
N_signal1 = length(signal1_mono); % Number of samples for Signal 1
N_signal2 = length(signal2_mono); % Number of samples for Signal 2

f_signal1 = (-N_signal1/2:N_signal1/2-1) * (Fs / N_signal1); % Frequency vector for
Signal 1
f_signal2 = (-N_signal2/2:N_signal2/2-1) * (Fs / N_signal2); % Frequency vector for
Signal 2

spectrum_signal1_original = fftshift(abs(fft(signal1_mono)) / N_signal1); % Spectrum
of Signal 1
spectrum_signal2_original = fftshift(abs(fft(signal2_mono)) / N_signal2); % Spectrum
of Signal 2


figure;
subplot(2, 1, 1);
plot(f_signal1, spectrum_signal1_original);
xlabel('Frequency (Hz)');
ylabel('Amplitude');
title('Spectrum of Original Signal 1');
grid on;

subplot(2, 1, 2);
plot(f_signal2, spectrum_signal2_original);
xlabel('Frequency (Hz)');
ylabel('Amplitude');
title('Spectrum of Original Signal 2');
grid on;

% Equalize signal lengths by zero-padding the shorter signal
max_length = max(length(signal1_mono), length(signal2_mono));
signal1_mono = [signal1_mono; zeros(max_length - length(signal1_mono), 1)];
signal2_mono = [signal2_mono; zeros(max_length - length(signal2_mono), 1)];


% Upsample the signals (increase sampling frequency to avoid aliasing)
upsample_factor = 10; % Upsampling factor
Fs_new = Fs * upsample_factor; % New sampling frequency after upsampling
signal1_upsampled = interp(signal1_mono, upsample_factor);
```

```
signal2_upsampled = interp(signal2_mono, upsample_factor);


% Define the time vector
N = length(signal1_upsampled); % Number of samples
t = (0:N-1) / Fs_new; % Time vector (in seconds)


% Define carrier frequencies
Fc1 = 100e3; % Carrier frequency for Signal 1 (in Hz)
Fc2 = Fc1 + 50e3; % Carrier frequency for Signal 2 (in Hz


% Generate carrier signals
carrier1 = cos(2 * pi * Fc1 * t'); % Carrier for Signal 1
carrier2 = cos(2 * pi * Fc2 * t'); % Carrier for Signal 2


% Perform DSB-SC modulation
modulated_signal1 = signal1_upsampled .* carrier1;
modulated_signal2 = signal2_upsampled .* carrier2;


% Compute Fourier Transforms for frequency domain analysis
f = (-N/2:N/2-1) * (Fs_new / N); % Frequency vector
spectrum_signal1 = fftshift(abs(fft(modulated_signal1)) / N); % Spectrum of Signal 1
spectrum_signal2 = fftshift(abs(fft(modulated_signal2)) / N); % Spectrum of Signal 2


% Plot modulated signals in the time domain
figure;
subplot(2, 1, 1);
plot(t, modulated_signal1);
xlabel('Time (s)');
ylabel('Amplitude');
title('Modulated Signal 1 in Time Domain');
grid on;

subplot(2, 1, 2);
plot(t, modulated_signal2);
xlabel('Time (s)');
ylabel('Amplitude');
title('Modulated Signal 2 in Time Domain');
grid on;

% Plot modulated signals in the frequency domain
figure;
subplot(2, 1, 1);
plot(f, spectrum_signal1);
xlabel('Frequency (Hz)');
ylabel('Amplitude');
title('Spectrum of Modulated Signal 1');
grid on;

subplot(2, 1, 2);
plot(f, spectrum_signal2);
```

```
xlabel('Frequency (Hz)');
ylabel('Amplitude');
title('Spectrum of Modulated Signal 2');
grid on;

% Combine the modulated signals to simulate the FDM signal
received_signal = modulated_signal1 + modulated_signal2;

% for the part of adding noise
% RF Stage: Design and apply band-pass filters (BPFs) for both signals
% Add AWGN to the input signals
%SNR = 10; % Signal-to-Noise Ratio in dB (adjust this to observe the effect of noise)
%noisy_signal = awgn(received_signal, SNR, 'measured'); % Add noise to Signal 1

% Band-pass filter parameters
BPF_bandwidth = 20e3; % Bandwidth of the BPF (20 kHz for audio signals)
BPF_order = 4; % Filter order

% Design BPF for Signal 1
BPF1 = designfilt('bandpassiir', 'FilterOrder', BPF_order, ...
                  'HalfPowerFrequency1', Fc1 - BPF_bandwidth/2, ...
                  'HalfPowerFrequency2', Fc1 + BPF_bandwidth/2, ...
                  'SampleRate', Fs_new);

% Design BPF for Signal 2
BPF2 = designfilt('bandpassiir', 'FilterOrder', BPF_order, ...
                  'HalfPowerFrequency1', Fc2 - BPF_bandwidth/2, ...
                  'HalfPowerFrequency2', Fc2 + BPF_bandwidth/2, ...
                  'SampleRate', Fs_new);

% Apply the BPF to the received signal for Signal 1
RF_signal1 = filter(BPF1, received_signal);

% Apply the BPF to the received signal for Signal 2
RF_signal2 = filter(BPF2, received_signal);

% Plot the filtered signals in the time domain
figure;
subplot(2, 1, 1);
plot(t, RF_signal1);
xlabel('Time (s)');
ylabel('Amplitude');
title('RF Signal 1 in Time Domain');
grid on;

subplot(2, 1, 2);
plot(t, RF_signal2);
xlabel('Time (s)');
ylabel('Amplitude');
title('RF Signal 2 in Time Domain');
grid on;

% Compute Fourier Transforms for frequency domain analysis
spectrum_filtered_signal1 = fftshift(abs(fft(RF_signal1)) / N);
spectrum_filtered_signal2 = fftshift(abs(fft(RF_signal2)) / N);
```

```
% Plot the filtered signals in the frequency domain
figure;
subplot(2, 1, 1);
plot(f, spectrum_filtered_signal1);
xlabel('Frequency (Hz)');
ylabel('Amplitude');
title('Spectrum of RF Signal 1');
grid on;

subplot(2, 1, 2);
plot(f, spectrum_filtered_signal2);
xlabel('Frequency (Hz)');
ylabel('Amplitude');
title('Spectrum of RF Signal 2');
grid on;


% Define IF frequency
Fc_IF = 25e3; % Intermediate frequency in Hz

% Add offset to oscillator frequencies
% (for the frequency offset requirement)
%offset = 1.2e3; % Frequency offset (case1: 0.2 kHz, case2: 1.2 KHz)
%to be added to oscillator_Fc1 and oscillator_Fc2

% Define oscillator frequency for Signal 1
oscillator_Fc1 = Fc1 + Fc_IF;
oscillator1 = cos(2 * pi * oscillator_Fc1 * t'); % Oscillator for Signal 1

% Mixing to IF for Signal 1
mixer_signal1 = RF_signal1 .* oscillator1;

% Define oscillator frequency for Signal 2
oscillator_Fc2 = Fc2 + Fc_IF ;
oscillator2 = cos(2 * pi * oscillator_Fc2 * t'); % Oscillator for Signal 2

% Mixing to IF for Signal 2
mixer_signal2 = RF_signal2 .* oscillator2;

% Plot the mixer signals in the time domain
figure;
subplot(2, 1, 1);
plot(t, mixer_signal1);
xlabel('Time (s)');
ylabel('Amplitude');
title('Mixer Signal 1 in Time Domain');
grid on;

subplot(2, 1, 2);
plot(t, mixer_signal2);
xlabel('Time (s)');
ylabel('Amplitude');
title('Mixer Signal 2 in Time Domain');
grid on;
```

```
% Compute and plot the spectrum of mixer signals
spectrum_mixer_signal1 = fftshift(abs(fft(mixer_signal1)) / N);
spectrum_mixer_signal2 = fftshift(abs(fft(mixer_signal2)) / N);

figure;
subplot(2, 1, 1);
plot(f, spectrum_mixer_signal1);
xlabel('Frequency (Hz)');
ylabel('Amplitude');
title('Spectrum of Mixer Signal 1');
grid on;

subplot(2, 1, 2);
plot(f, spectrum_mixer_signal2);
xlabel('Frequency (Hz)');
ylabel('Amplitude');
title('Spectrum of Mixer Signal 2');
grid on;


% IF Stage: Band-pass filter centered at Fc_IF for both signals
IF_BPF_order = 4;
IF_BPF_bandwidth = 10e3; % Bandwidth for the IF stage filter (10 kHz)

% Design IF BPF
IF_BPF = designfilt('bandpassiir', 'FilterOrder', IF_BPF_order, ...
                    'HalfPowerFrequency1', Fc_IF - IF_BPF_bandwidth/2, ...
                    'HalfPowerFrequency2', Fc_IF + IF_BPF_bandwidth/2, ...
                    'SampleRate', Fs_new);

% Apply IF BPF
IF_signal1 = filter(IF_BPF, mixer_signal1);
IF_signal2 = filter(IF_BPF, mixer_signal2);

% Plot the IF filtered signals in the time domain
figure;
subplot(2, 1, 1);
plot(t, IF_signal1);
xlabel('Time (s)');
ylabel('Amplitude');
title('IF Signal 1 in Time Domain');
grid on;

subplot(2, 1, 2);
plot(t, IF_signal2);
xlabel('Time (s)');
ylabel('Amplitude');
title('IF Signal 2 in Time Domain');
grid on;

% Compute Fourier Transforms for IF filtered signals
spectrum_IF_signal1_filtered = fftshift(abs(fft(IF_signal1)) / N);
spectrum_IF_signal2_filtered = fftshift(abs(fft(IF_signal2)) / N);
```

```
% Plot the IF filtered signals in the frequency domain
figure;
subplot(2, 1, 1);
plot(f, spectrum_IF_signal1_filtered);
xlabel('Frequency (Hz)');
ylabel('Amplitude');
title('Spectrum of IF Signal 1');
grid on;

subplot(2, 1, 2);
plot(f, spectrum_IF_signal2_filtered);
xlabel('Frequency (Hz)');
ylabel('Amplitude');
title('Spectrum of IF Signal 2');
grid on;

% Baseband Detection
% Generate local oscillator for baseband mixing
local_oscillator1 = cos(2 * pi * Fc_IF * t'); % Local oscillator for Signal 1
local_oscillator2 = cos(2 * pi * Fc_IF * t'); % Local oscillator for Signal 2

% Mix down to baseband
baseband_signal1 = IF_signal1 .* local_oscillator1;
baseband_signal2 = IF_signal2 .* local_oscillator2;

% Plot the baseband signals in the time domain
figure;
subplot(2, 1, 1);
plot(t, baseband_signal1);
xlabel('Time (s)');
ylabel('Amplitude');
title('Baseband Signal 1 in Time Domain');
grid on;

subplot(2, 1, 2);
plot(t, baseband_signal2);
xlabel('Time (s)');
ylabel('Amplitude');
title('Baseband Signal 2 in Time Domain');
grid on;

% Compute Fourier Transforms for baseband signals
spectrum_baseband_signal1 = fftshift(abs(fft(baseband_signal1)) / N);
spectrum_baseband_signal2 = fftshift(abs(fft(baseband_signal2)) / N);

% Plot the baseband signals in the frequency domain
figure;
subplot(2, 1, 1);
plot(f, spectrum_baseband_signal1);
xlabel('Frequency (Hz)');
ylabel('Amplitude');
title('Spectrum of Baseband Signal 1');
grid on;

subplot(2, 1, 2);
plot(f, spectrum_baseband_signal2);
```

```
xlabel('Frequency (Hz)');
ylabel('Amplitude');
title('Spectrum of Baseband Signal 2');
grid on;

% Design LPF for baseband recovery
LPF_cutoff = 15e3; % LPF cutoff frequency (15 kHz for audio signals)
LPF_order = 4;
LPF = designfilt('lowpassiir', 'FilterOrder', LPF_order, ...
                 'HalfPowerFrequency', LPF_cutoff, ...
                 'SampleRate', Fs_new);

% Apply LPF to extract baseband signals
recovered_signal1 = filter(LPF, baseband_signal1);
recovered_signal2 = filter(LPF, baseband_signal2);

% Downsample the recovered signals to the original sampling rate
recovered_signal1_downsampled = downsample(recovered_signal1, upsample_factor);
recovered_signal2_downsampled = downsample(recovered_signal2, upsample_factor);

% Normalize the recovered signals
recovered_signal1_downsampled = recovered_signal1_downsampled /
max(abs(recovered_signal1_downsampled));
recovered_signal2_downsampled = recovered_signal2_downsampled /
max(abs(recovered_signal2_downsampled));

% Plot the recovered signals in the time domain
t_recovered = (0:length(recovered_signal1_downsampled)-1) / Fs;

figure;
subplot(2, 1, 1);
plot(t_recovered, recovered_signal1_downsampled);
xlabel('Time (s)');
ylabel('Amplitude');
title('Recovered Signal 1 in Time Domain');
grid on;

subplot(2, 1, 2);
plot(t_recovered, recovered_signal2_downsampled);
xlabel('Time (s)');
ylabel('Amplitude');
title('Recovered Signal 2 in Time Domain');
grid on;

% Compute Fourier Transforms for recovered signals
spectrum_recovered_signal1 = fftshift(abs(fft(recovered_signal1)) / N);
spectrum_recovered_signal2 = fftshift(abs(fft(recovered_signal2)) / N);

% Plot the recovered signals in the frequency domain
figure;
subplot(2, 1, 1);
plot(f, spectrum_recovered_signal1);
xlabel('Frequency (Hz)');
ylabel('Amplitude');
title('Spectrum of Recovered Signal 1');
grid on;
```

```
subplot(2, 1, 2);
plot(f, spectrum_recovered_signal2);
xlabel('Frequency (Hz)');
ylabel('Amplitude');
title('Spectrum of Recovered Signal 2');
grid on;

% Play only the recovered Signal 1
disp('Playing Recovered Signal 1...');
sound(recovered_signal1_downsampled, Fs);
pause(length(recovered_signal1_downsampled) / Fs + 1); % Wait for playback to finish
```